Project

Mobile Application Development

dr inż. Radosław Maciaszczyk
Faculty of Computer Science
West Pomeranian University of Technology, Szczecin

Overview

This part of the **Mobile Application Development** course aims to build an Android application based on your own or the instructor's suggested topic. The project consists of two elements: **documentation** and a **created application**. All applications must be written in **Kotlin**.

Assessment

Element	**Points**
Documentation	up to 10 pts
Basic application	20–30 pts
Medium application	25–35 pts
Advanced application	30–45 pts
Maximum score	**55 pts**
+ Tests bonus	**+5 pts**

Documentation Requirements

The documentation shall consist of the following elements. Items 1–4 are completed during the **first two classes**; items 5–7 are completed **ongoing** during application development.

1. Brief description of the purpose of the project
2. List of functional requirements
3. List of non-functional requirements
4. Layout design
5. List of libraries used
6. Specification of the permissions used and a description of the functionality for which each permission was used
7. Description of testing: manual test cases (scenario, input, expected result) and automated tests if applicable

Complexity Requirements

I. Basic Application (20–30 pts)

Requirements:

- Consists of several activities / fragments
- Requires the use of **one sensor** or a **database**
- UI: XML layouts or **Jetpack Compose**
- Recommended architecture: **ViewModel** + **StateFlow** (Jetpack)

Example applications:

- Application to draw by shaking the phone (with settings screen and drawing history)
- Application for memorising encrypted notes
- Puzzle game with result history
- Step counter using the device step sensor, with daily goal setting and weekly history stored in a local database

II. Medium Application (25–35 pts)

Requirements:

- Several activities / fragments
- Use of **one sensor** or **database** is required
- Requires use of **internet connection**, e.g. via API or GNSS system
- UI: **Jetpack Compose** recommended
- Required architecture: **ViewModel** + **Repository** + **StateFlow** (Jetpack)
- Dependency injection: **Hilt** recommended
- Use of **Google services** is recommended, e.g. Maps

Example applications:

- Application for memorising located notes, search by specifying the area on the map
- Generation of captions on photographs based on EXIF data, with place name resolved via **Google Maps Geocoding API** (e.g. date and location of the photo)
- Application using NASA's API, e.g. for displaying "Astronomy Picture of Day" photos with caching of queries and responses
- Weather application: current conditions and forecast based on current position (GNSS) or searched city, with history of recently searched locations (weather data via public API, location via **Google Maps**)

III. Advanced Application (30–45 pts)

Requirements:

- Several activities / fragments
- Use of **one sensor** or **database** is required
- Requires use of **web connection**, e.g. via API or GNSS system
- UI: **Jetpack Compose** required
- Required architecture: **ViewModel** + **Repository** + **StateFlow** (Jetpack)
- Dependency injection: **Hilt** required
- The use of **Google services is required**, e.g. Maps or cloud databases

Example applications:

- Real-time location sharing between users displayed on **Google Maps**, with route history stored in **Firebase**
- Interactive multiplayer game with real-time data synchronisation and match history in **Firebase Realtime Database**
- Application for accounting for shared expenses with cloud data synchronisation (**Firebase**), push notifications and group management
- Fitness tracker: GPS route recording displayed on **Google Maps**, activity statistics, workout history stored in the cloud

Complexity Comparison

Requirement	Basic	Medium	Advanced
Several activities / layouts	✓	✓	✓
Sensor or database	one of	required	required
Internet / API / GNSS	–	required	required
Jetpack Compose (UI)	optional	recommended	required
ViewModel + StateFlow (Jetpack)	recommended	required	required
Repository pattern	–	required	required
Hilt (dependency injection)	–	recommended	required
Google services	–	recommended	required
Points	20–30	25–35	30–45

Submission

- **Deadline:** 16 June 2026
- **Form:** source code submitted via **GitHub** repository (link sent to the instructor)
- The repository must contain: application source code, documentation (PDF or Markdown), and a short `README.md` with build instructions

- A brief **live demo** (5–10 min) during the last class is required

Grading Scale

Points	Grade	Description
below 30	2.0	Fail
30–34	3.0	Pass
35–39	3.5	Satisfactory
40–44	4.0	Good
45–49	4.5	Very good
50+	5.0	Excellent

Bonus points:

- **+5 pts** – application includes **automated tests**:
  - unit tests covering ViewModel / Repository logic (e.g. JUnit + MockK)
  - and/or UI instrumented tests (e.g. Espresso or Compose UI Test)
  - minimum coverage: **>60%** of ViewModel / Repository code

Project – Class 1

Deadline: 24 March 2026

Prepare a brief description of your planned application and submit it as a **PDF file**.

The description should include the following:

1. **Project name**
   A short, meaningful name for your application.

2. **Functions**
   List the main functionalities of the application (what the user can do). Use the form of functional requirements, e.g.:

   - The user can ...
   - The application displays ...
   - The system stores ...

3. **Used sensors, hardware, and APIs**
   Specify all external resources the application will use, e.g.:

   - Sensors: accelerometer, GPS, camera, step counter, ...
   - Hardware: Bluetooth, NFC, ...
   - External APIs: Google Maps, Firebase, REST APIs, ...
   - Local database: Room, ...

4. **Complexity level**
   Declare the intended complexity level: **Basic** / **Medium** / **Advanced**
   Justify briefly why the application meets the requirements of the chosen level.

> **Note:** The description does not have to be final – it may be updated during the course. However, the complexity level and core functionality should be decided at this stage and any major changes must be agreed with the instructor.

Project – Class 1

Sample description

---

1. Project name

WeatherNow

2. Functions

- The user can check current weather conditions for their current location (auto-detected via GPS)
- The user can search for weather by city name
- The application displays: temperature, humidity, wind speed, weather icon, and a 5-day forecast
- The application shows the searched location on a Google Maps minimap
- The system stores the history of the last 10 searched locations in a local database
- The user can mark locations as favourites and access them from the home screen
- The application refreshes data automatically when opened

3. Used sensors, hardware, and APIs

Category	Details
Sensor	GPS / GNSS – current device location
External API	OpenWeatherMap REST API – weather data and 5-day forecast
Google services	Google Maps SDK – location display; Google Maps Geocoding API – city name ↔ coordinates
Local database	Room – search history and favourites

4. Complexity level

Medium

Justification:

- Several screens: Home, Search, History, Favourites, Detail ($\geq$ 4 fragments)
- Database required ✓ (Room – history and favourites)
- Internet connection required ✓ (OpenWeatherMap API + Google Maps)
- Architecture: ViewModel + Repository + StateFlow ✓
- Google services used ✓ (Maps SDK + Geocoding API)